==========

==========

# CHAPTER 3 — TTBR, ASID, AND PROCESS ISOLATION

==========

==========

***A complete, maximum-detail breakdown of TTBR registers, ASID tagging, context switching, and process isolation in ARMv8.***

**********

**********

## 3.1 — Introduction to Translation Table Base Registers

**********

**********

ARMv8 uses **Translation Table Base Registers (TTBRs)** to define the root of the Stage-1 translation tables:

- **TTBR0_EL1** — typically user space

- **TTBR1_EL1** — typically kernel space

Each TTBR contains:

- a physical base address
- memory attribute overrides
- ASID (TTBR0 only)
- control bits

TTBRs determine where the MMU begins the page table walk.

**********

**********

# 3.2 — TTBR0_EL1 (User Space Root)

**********

**********

TTBR0_EL1 is used for:

- EL0 user processes
- per-process virtual memory
- ASID-tagged TLB entries

### TTBR0_EL1 Fields

| Field | Purpose |
|---|---|
| **Base address** | Root of Stage-1 tables |
| **ASID** | Process identifier |
| **RGN** | Cacheability override |
| **SH** | Shareability override |

| | |
|---|---|
| **IRGN** | Inner cacheability override |

### Teaching Note

TTBR0_EL1 changes on **every context switch**.

**********

**********

# 3.3 — TTBR1_EL1 (Kernel Space Root)

**********

**********

TTBR1_EL1 is used for:

- EL1 kernel mappings
- global mappings
- supervisor memory
- exception vectors
- kernel stacks
- kernel code/data

### TTBR1 Characteristics

- **Global** — shared across all processes
- **Not ASID-tagged**
- **Never switched during context switches**
- **Always resident in TLB** (high hit rate)

### Implementation Note

Kernel memory must be mapped in TTBR1 to avoid flushing on context switches.

**********

**********

# 3.4 — TTBR0/TTBR1 Split

**********

**********

The split between TTBR0 and TTBR1 is controlled by:

- **TCR_EL1.T0SZ**
- **TCR_EL1.T1SZ**

These define how much of the VA space belongs to each TTBR.

### Example Split (48-bit VA)

```
TTBR0: 0x0000_0000_0000_0000 → 0x0000_FFFF_FFFF_FFFF
TTBR1: 0xFFFF_0000_0000_0000 → 0xFFFF_FFFF_FFFF_FFFF
```

### Teaching Note

TTBR1 typically maps kernel memory at the top of the VA space.

**********

**********

## 3.5 — TTBR Selection Logic

**********

**********

The MMU selects TTBR based on VA:

```
if VA in TTBR0 region:
    use TTBR0_EL1
else:
    use TTBR1_EL1
```

### Implementation Note

TTBR selection happens **before** any TLB lookup.

**********

**********

## 3.6 — TTBR Memory Attribute Overrides

**********

**********

TTBRs contain override bits:

- **IRGN** — Inner cacheability
- **ORGN** — Outer cacheability
- **SH** — Shareability

These override descriptor attributes **only if enabled**.

### Example

TTBR0_EL1.IRGN = Write-Back → All Stage-1 tables accessed through TTBR0 use WB caching.

**********

**********

# 3.7 — TTBR Physical Address Requirements

**********

**********

TTBR base addresses must be:

- aligned to page table size
- physical addresses (not virtual)
- within valid PA range (defined by ID_AA64MMFR0_EL1.PARange)

### Alignment Example (4 KB granule)

```
TTBR base must be 4 KB aligned.
```

**********

**********

# 3.8 — ASID (Address Space Identifier)

**********

**********

ASID is stored in **TTBR0_EL1[63:48]**.

ASID identifies:

- which process owns a TLB entry
- which TTBR0 mappings are valid
- which entries must be invalidated on context switch

### ASID Size

ARMv8 supports:

- **8-bit ASID** (256 processes)
- **16-bit ASID** (65,536 processes)

### Teaching Note

ASID allows multiple processes to coexist in the TLB without flushing.

**********

**********

# 3.9 — ASID and TLB Tagging

**********

**********

TLB entries contain:

- VA
- PA
- ASID
- permissions
- attributes

TLB lookup requires:

Code
```
VA + ASID match
```

### Implementation Note

Kernel mappings (TTBR1) are **global** and do not use ASID.

**********

**********

# 3.10 — ASID Allocation Strategy

**********

**********

OS must allocate ASIDs carefully:

## Common Strategies

1. **Simple Incremental Allocation**
    a. ASID++ per process
    b. Wrap around when full
    c. Requires TLB flush on wrap
2. **Generation-Based Allocation**
    a. ASID = (generation << bits) | process_id
    b. Avoids flushing entire TLB
    c. Used in Linux and QNX
3. **Static Partitioning**
    a. ASID ranges assigned to subsystems
    b. Used in RTOS environments

## Teaching Note

ASID reuse without flushing causes catastrophic memory corruption.

**********

**********

# 3.11 — ASID and Context Switching

**********

**********

During context switch:

1. TTBR0_EL1 updated with new base address
2. TTBR0_EL1 updated with new ASID
3. TLB entries for old ASID remain cached
4. New ASID ensures isolation

### Implementation Note

TLBI instructions are only needed when ASID is reused.

**********

**********

# 3.12 — ASID and Kernel Isolation

**********

**********

Kernel mappings use TTBR1_EL1 and are:

- global
- not ASID-tagged
- shared across all processes

This ensures:

- kernel code/data always accessible
- high TLB hit rate
- no flushes on context switch

**********

**********

# 3.13 — ASID and Hypervisor Interaction

**********

**********

Hypervisor uses **VMID** for Stage-2 TLB entries.

ASID and VMID combine to isolate:

- processes
- VMs
- nested virtualization environments

### Combined Tag

TLB entries are tagged with:

```
ASID + VMID
```

This allows:

- multiple VMs
- each with multiple processes
- all coexisting in TLB

**********

**********

# 3.14 — TTBR0_EL2 (Stage-2 Root)

**********

**********

TTBR0_EL2 defines the root of Stage-2 translation tables.

Used for:

- IPA → PA translation
- VM isolation
- hypervisor-controlled permissions
- hypervisor-controlled memory attributes

### Teaching Note

TTBR0_EL2 is the hypervisor's most important register.

**********

**********

# 3.15 — TTBR and Speculation

**********

**********

Speculative execution may:

- read TTBR values
- begin speculative page table walks
- fill TLB entries speculatively

### Implementation Note

Use barriers (DSB/ISB) when updating TTBRs to prevent speculative use of stale values.

**********

**********

# 3.16 — TTBR and Memory Ordering

**********

**********

TTBR updates must be followed by:

- **DSB** — ensure visibility

- **ISB** — ensure pipeline flush

Example:

```
MSR TTBR0_EL1, x0
DSB SY
ISB
```

**********

**********

# 3.17 — TTBR and Kernel ASLR

**********

**********

Kernel Address Space Layout Randomization (KASLR) may:

- randomize TTBR1_EL1 base
- randomize kernel VA layout
- require dynamic TTBR1 updates

### Teaching Note

TTBR1 updates must follow full barrier sequence.

**********

**********

## 3.18 — TTBR and User ASLR

**********

**********

User ASLR randomizes:

- TTBR0_EL1 base
- user stack
- heap
- code segments

ASID ensures isolation even with randomized TTBR0.

**********

**********

## 3.19 — TTBR and Memory Partitioning (QNX-Style)

**********

**********

QNX uses memory partitions:

- system partition
- process partition
- I/O partition

TTBR0/TTBR1 mapping aligns with partition boundaries.

### Implementation Note

QNX may use ASID ranges to isolate partitions.

**********

**********

# 3.20 — TTBR Fault Conditions

**********

**********

Faults occur if:

- TTBR base is misaligned
- TTBR points to invalid memory
- TTBR points to unmapped PA
- TTBR points to Device memory incorrectly

Fault type: **Translation Fault**.

**********

**********

## 3.21 — Chapter Summary

**********

**********

This chapter covered:

- TTBR0/TTBR1 roles
- TTBR selection logic
- TTBR memory attribute overrides
- ASID tagging
- ASID allocation strategies
- ASID + VMID combined isolation
- TTBR0_EL2 for Stage-2
- TTBR updates and barriers
- TTBR interaction with ASLR
- TTBR fault conditions